

Shell Commands Reference

Linux / WSL · PowerShell · Command Prompt
Complete Guide for the OnDevice HAR FQT+RigL Project

1. What Is a Shell?

A **shell** is a program that lets you type text commands to control your computer. Different environments use different shells with different syntax. This project uses three of them.

Shell	Where it runs	Command separator	Best for
Linux / WSL (bash)	Ubuntu inside Windows (WSL2)	&& ;	C builds, Python, native Linux tools
PowerShell	Windows blue terminal or VS Code	;	Windows automation, file management
Command Prompt (CMD)	Windows black terminal (cmd.exe)	& &&	Old Windows scripts, basic file ops

Key rule: && chains two commands (run second only if first succeeds) in bash and CMD, but it is **NOT valid in PowerShell**. Use ; in PowerShell or put commands on separate lines.

2. Identifying Your Shell (the Prompt)

Look at the text that appears before your cursor — that is the prompt. Each shell has a distinct look:

```
matef@PC:~$ <- Linux / WSL (ends with $ or #)
PS C:\Users\matef> <- PowerShell (starts with PS)
C:\Users\matef> <- Command Prompt (just a path>)
```

How to open it	Linux / WSL	PowerShell	Command Prompt
Start menu / search	Search 'WSL' or type wsl in any terminal	Search 'PowerShell' or press Win+X -> Terminal	Search 'cmd' or press Win+R, type cmd, Enter

3. Navigation Commands

Moving between folders is the most common task. Note: Linux uses forward slashes (/) and Windows uses backslashes (\).

Task	Linux / WSL	PowerShell	Command Prompt
------	-------------	------------	----------------

Print current path	pwd	pwd or Get-Location	cd (no argument)
Go into a folder	cd foldername	cd foldername	cd foldername
Go up one level	cd ..	cd ..	cd ..
Go to home folder	cd ~	cd \$HOME	cd %USERPROFILE%
Go to Windows drive (from WSL)	cd /mnt/f/...	cd F:\... (already on Windows)	F: cd ...
List files	ls or ls -la	ls or dir	dir

Example — navigating to the project folder

Linux / WSL

```
cd "/mnt/f/technical material/Master thesis/claude/on_device training"
cd OnDeviceTraining_HAR_FQT_RigL
```

PowerShell

```
cd "F:\technical material\Master thesis\claude\on_device training"
cd OnDeviceTraining_HAR_FQT_RigL
```

Command Prompt

```
cd "F:\technical material\Master thesis\claude\on_device training"
cd OnDeviceTraining_HAR_FQT_RigL
```

4. File and Folder Operations

Task	Linux / WSL	PowerShell	Command Prompt
Create a folder	<code>mkdir foldername</code>	<code>mkdir foldername</code>	<code>mkdir foldername</code>
Delete folder + contents	<code>rm -rf foldername</code>	<code>Remove-Item -Recurse -Force foldername</code>	<code>rmdir /s /q foldername</code>
Delete a single file	<code>rm file.txt</code>	<code>Remove-Item file.txt</code>	<code>del file.txt</code>
Rename a file	<code>mv old.txt new.txt</code>	<code>Rename-Item old.txt new.txt</code>	<code>ren old.txt new.txt</code>
Copy a file	<code>cp src.txt dst.txt</code>	<code>Copy-Item src.txt dst.txt</code>	<code>copy src.txt dst.txt</code>
View file contents	<code>cat file.txt</code>	<code>Get-Content file.txt</code>	<code>type file.txt</code>
Chain commands (if first succeeds)	<code>cmd1 && cmd2</code>	<code>cmd1 ; cmd2 (NOT &&)</code>	<code>cmd1 && cmd2</code>
Run commands in sequence	<code>cmd1 ; cmd2</code> or <code>cmd1 && cmd2</code>	<code>cmd1 ; cmd2</code>	<code>cmd1 & cmd2</code>

Why && fails in PowerShell

PowerShell treats && as a bitwise AND operator, not a command separator. This causes a parser error. The fix is simple: use ; instead, or put each command on its own line.

WRONG — causes error in PowerShell

```
mkdir build_pc && cd build_pc
```

Note: The token '&&' is not a valid statement separator in this version.

CORRECT — PowerShell

```
mkdir build_pc
cd build_pc
# OR on one line:
mkdir build_pc ; cd build_pc
```

5. WSL Commands (typed in PowerShell or CMD)

WSL (Windows Subsystem for Linux) runs a full Linux environment inside Windows. These commands control WSL itself and are typed in PowerShell or CMD.

Command	What it does
<code>wsl</code>	Open a WSL bash terminal
<code>wsl --list</code>	Show all installed Linux distributions
<code>wsl --shutdown</code>	Stop all running WSL instances – releases file locks so you can delete folders that WSL has open

wsl --version	Show WSL and kernel version information
---------------	---

Windows paths inside WSL

Windows drives are mounted at /mnt/. Drive F:\ becomes /mnt/f/.

```
# Windows path: F:\technical material\Master thesis\...
# WSL bash path: /mnt/f/technical material/Master thesis/...

# Navigate there inside WSL:
cd "/mnt/f/technical material/Master thesis/claude/on_device training"
```

Why we build inside WSL instead of Windows

The OnDeviceTraining framework requires GCC 10 or newer. The MinGW GCC 8.1 that comes bundled with CodeBlocks is too old — it rejects certain valid C code patterns used by the framework. WSL ships with Ubuntu's GCC (12+) which handles these correctly. Additionally, the Windows DrvFS filesystem prevents some file operations (mkdir, rm) from WSL on Windows-mounted paths, so we build in a native Linux path (~/.har_build) instead.

6. Installing Tools in WSL

WSL uses **apt** (Advanced Package Tool) to install software. Run commands one at a time and wait for each to finish.

Linux / WSL — install all required tools

```
# Step 1: update the package list
sudo apt update

# Step 2: install cmake, gcc, g++, make
sudo apt install -y cmake gcc g++ make

# Step 3: verify
cmake --version
gcc --version
```

`sudo` = 'superuser do' — runs the command as administrator. WSL will ask for your Linux password the first time.

The `-y` flag means 'yes to all prompts' so you do not have to confirm each package installation manually.

Do NOT try to install packages one per line in the same command — paste each command separately and wait for completion.

7. CMake Build Commands

CMake reads `CMakeLists.txt` and generates the actual build files. You always do two steps: **configure** (`cmake`) then **build** (`cmake --build .`).

Step	Linux / WSL (recommended)	PowerShell / CMD
Create build folder	<code>mkdir -p ~/har_build cd ~/har_build</code>	<code>mkdir build_pc cd build_pc</code>
Configure (run once per clean build)	<code>cmake "/mnt/f/.../project_folder"</code>	<code>cmake ..</code> (must be in build folder)
Build (compile)	<code>cmake --build .</code>	<code>cmake --build . --config Release</code>
Clean and rebuild	<code>cd ~ rm -rf har_build mkdir har_build cd har_build</code>	<code>Remove-Item -Recurse build_pc mkdir build_pc cd build_pc</code>

Important: CMakeLists.txt must be named exactly that

CMake always looks for a file named exactly `CMakeLists.txt`. There is no `-f` flag to specify a different filename (unlike Make). If your file is named `CMakeLists_PC.txt`, rename it before running `cmake`.

PowerShell — rename files

```
# Rename the Pico CMake file out of the way:
Rename-Item CMakeLists.txt CMakeLists_Pico.txt

# Rename the PC cmake file to the standard name:
Rename-Item CMakeLists_PC.txt CMakeLists.txt
```

Command Prompt — rename files

```
ren CMakeLists.txt CMakeLists_Pico.txt
ren CMakeLists_PC.txt CMakeLists.txt
```

8. Complete Workflow: Build and Run This Project

Follow these steps in order from a fresh WSL terminal. Each step must complete without errors before continuing.

Step 1 — Install build tools (once only)

```
sudo apt update
sudo apt install -y cmake gcc g++ make
cmake --version # should print 3.x.x
```

Step 2 — Create build folder and configure

```
mkdir -p ~/har_build
cd ~/har_build
cmake "/mnt/f/technical material/Master thesis/claude/on_device
training/OnDeviceTraining_HAR_FQT_Rigl"
# Should end with: -- Build files have been written to: /home/matef/har_build
```

Step 3 — Compile the project

```
cmake --build .  
# When done: [100%] Built target har_fqt_rigl
```

Step 4 — Generate dataset (once only)

```
# Navigate to OnDeviceTraining root first:  
cd "/mnt/f/technical material/Master thesis/claude/on_device training/OnDeviceTraining"  
python3 examples/har_classifier/prepare_data.py  
# Creates: train_x.npy, train_y.npy, val_x.npy, val_y.npy, test_x.npy, test_y.npy
```

Step 5 — Run the training

```
# Run from the OnDeviceTraining root so data paths resolve:  
cd "/mnt/f/technical material/Master thesis/claude/on_device training/OnDeviceTraining"  
~/har_build/har_fqt_rigl  
  
# Or pass the data directory explicitly:  
~/har_build/har_fqt_rigl "examples/har_classifier/data"
```

Press Ctrl+C to stop the training at any time.

9. Common Errors and Fixes

Error	Cause	Fix
&& is not a valid statement separator	Used bash syntax in PowerShell	Replace && with ; or use separate lines
cmake: not found	CMake not installed in WSL	sudo apt install -y cmake
Error: not a CMake build directory (no CMakeCache.txt)	Not in build folder, or cmake was not run yet	cd ~/har_build then cmake
CMake Error: Unknown argument -f	CMake has no -f flag for custom filenames	Rename file to CMakeLists.txt before running cmake
The process cannot access the file (used by another process)	WSL has a lock on the Windows folder	wsl --shutdown in CMD, then delete the folder
Permission denied (rm or mkdir from WSL)	DrvFS limits ops on Windows-mounted paths	Build in native Linux: mkdir -p ~/har_build
GCC error: a label can only be part of a statement	MinGW GCC 8.1 is too old (CodeBlocks default)	Use WSL with GCC 10+ (install via apt)
Cannot open .npy file	Data not generated yet or wrong path	Run prepare_data.py first, then check the path

10. Quick Reference Card

Task	Linux / WSL	PowerShell	CMD
Go to project	cd /mnt/f/...	cd F:\...	cd F:\...
List files	ls	ls or dir	dir
Make folder	mkdir name	mkdir name	mkdir name
Delete folder	rm -rf name	Remove-Item -Recurse -Force name	rmdir /s /q name
Rename file	mv old new	Rename-Item old new	ren old new
Configure cmake	cmake /path/to/src	cmake ..	cmake ..
Build project	cmake --build .	cmake --build . --config Release	cmake --build .
Stop program	Ctrl+C	Ctrl+C	Ctrl+C
Open WSL	(already in WSL)	wsl	wsl
Shutdown WSL	(from Windows) wsl --shutdown	wsl --shutdown	wsl --shutdown
Chain commands	cmd1 && cmd2	cmd1 ; cmd2	cmd1 && cmd2

Install package	sudo apt install pkg	(not available)	(not available)
-----------------	----------------------	-----------------	-----------------